

# LAB MANUAL: Image Processing in Transform Domain (Fourier Transform)

## Objective

- Understand Transform Domain Processing
- Apply Fourier Transform on an image
- Apply Low Pass and High Pass Filters
- Perform Inverse Transform
- Analyze Frequency Spectrum

## Step 1: Import Libraries

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Working:

cv2 → Used to read and process images.

numpy → Used for mathematical operations and Fourier Transform.

matplotlib → Used to display images and graphs.

## Step 2: Read Image

Code:

```
img = cv2.imread('image.jpg', 0)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.show()
```

Working:

cv2.imread(..., 0) reads the image in grayscale.

imshow() displays the image.

cmap='gray' ensures it shows as grayscale.

show() renders the image output.

## Step 3: Apply Fourier Transform

Code:

```
f = np.fft.fft2(img)
```

```
fshift = np.fft.fftshift(f)
```

Working:

fft2() converts image from spatial domain to frequency domain.  
fftshift() shifts low frequencies to the center of the spectrum for better visualization.

## Step 4: Display Magnitude Spectrum

Code:

```
magnitude_spectrum = 20*np.log(np.abs(fshift))  
plt.imshow(magnitude_spectrum, cmap='gray')  
plt.title("Frequency Spectrum")  
plt.show()
```

Working:

np.abs() calculates magnitude of complex numbers.  
np.log() enhances visibility of spectrum.  
Low frequencies appear at center.  
High frequencies appear at edges.

## Step 5: Low Pass Filter

Code:

```
rows, cols = img.shape  
crow, ccol = rows//2 , cols//2  
mask = np.zeros((rows, cols), np.uint8)  
mask[crow-30:crow+30, ccol-30:ccol+30] = 1  
fshift_filtered = fshift * mask
```

Working:

Creates a square mask in the center.  
Only low frequencies are kept.  
High frequencies are removed.  
Image becomes blurred after inverse transform.

## Step 6: Inverse Transform (Low Pass Result)

Code:

```
f_ishift = np.fft.ifftshift(fshift_filtered)  
img_back = np.fft.ifft2(f_ishift)  
img_back = np.abs(img_back)
```

Working:

ifftshift() shifts spectrum back.  
ifft2() converts frequency domain back to spatial domain.

`np.abs()` removes imaginary values.  
Result = Blurred Image.

## Step 7: High Pass Filter

Code:

```
mask = np.ones((rows, cols), np.uint8)
mask[crow-30:crow+30, ccol-30:ccol+30] = 0
fshift_filtered = fshift * mask
```

Working:

Removes low frequencies from center.  
Keeps high frequency components.  
Enhances edges and fine details.

## Step 8: Inverse Transform (High Pass Result)

Code:

```
f_ishift = np.fft.ifftshift(fshift_filtered)
img_back = np.fft.ifft2(f_ishift)
img_back = np.abs(img_back)
```

Working:

Performs inverse transform.  
Final output highlights edges.  
Useful for crack detection and edge enhancement.

## Conclusion

Transform Domain Processing converts an image to frequency domain, modifies frequency components, and converts back to spatial domain.

Low Pass Filter → Smooth / Blur image

High Pass Filter → Enhance edges

Applications:

- Noise removal
- Image enhancement
- Crack detection
- Image compression